QA Evidence — Server Security Response Headers (Item O)

Revision: 1 **Last modified:** 2026-07-09T20:40:26Z **Scope:** `server/` (Gin control plane — API + embedded OTA-Manager SPA) **Stream:** O-impl (implementation subagent) **Authority:** operator-approved all tiers incl. SPA-document CSP; brief `docs/research/operator_brief_security_headers_ddos_20260710/BRIEF.md` **Anti-bluff:** §11.4.108 (runtime-signature), §11.4.123 (rock-solid proof), §11.4.170 (real-UI compatibility), §11.4.5/§11.4.69 (captured evidence)

---

0. Summary verdict

Tiered security response headers implemented and wired into the real router chain. Every PASS below cites a captured test log run against the REAL `Server.Router()` (global chain + API group + `MountManagerUI`), not a stand-in. The SPA-document CSP is proven compatible with the REAL built Vite/React manager bundle (embed is a real build, not a placeholder). `go build`, `go vet`, `go test -race` all green.

This change is **purely additive** — it adds response headers; it removes no capability and does not touch `HELIX_MAX_INFLIGHT` (a separate item).

---

1. Files created / modified

File	Change
`server/internal/api/security_headers.go`	**NEW** — Tier A/B middleware, Tier C SPA-CSP builder, `permissionsPolicyDenyAll`, `hstsValue`, `apiCSP`, `spaCSPBase`, `spaDocumentCSP(tlsEnabled)`, `(*Server).tlsEnabled()`
`server/internal/api/security_headers_test.go`	**NEW** — 8 tests (Tier A on every response class incl. 429 shed; HSTS TLS-gating; Tier B

File	Change
	scoping; Tier C document CSP + UIR gating; real-bundle CSP-compatibility gate)
`server/internal/api/server.go`	**MOD** — wired `securityHeadersMiddleware(s.tlsEnabled())` into the global chain (before the in-flight limiter); wired `apiSecurityHeadersMiddleware()` onto the `v1` API group
`server/internal/api/embed.go`	**MOD** — `MountManagerUI` sets the Tier-C SPA document CSP via a `/manager` group middleware + the NoRoute client-route fallback

2. Headers implemented, per tier

Tier A — global (`securityHeadersMiddleware`, EVERY response)

Set at `server.go` chain position `recovery → requestID → securityHeaders → maxInflight → compression`, so the headers are present on success, error, panic (500), and 429-shed responses.

Header	Value
`X-Content-Type-Options`	`nosniff`
`X-Frame-Options`	`DENY`
`Referrer-Policy`	`no-referrer`
`Cross-Origin-Opener-Policy`	`same-origin`
`Cross-Origin-Resource-Policy`	`same-origin`
`Permissions-Policy`	deny-all (`accelerometer=(), … serial=()` — every feature `()`)
`X-Permitted-Cross-Domain-Policies`	`none`
`Strict-Transport-Security`	`max-age=63072000; includeSubDomains` — **only when TLS configured**

`Cross-Origin-Embedder-Policy: require-corp` **deliberately OMITTED** (high breakage risk, no benefit — per brief §1.3.2). `preload` omitted from HSTS (hard-to-reverse commitment).

Tier B — API group only (`apiSecurityHeadersMiddleware`, under `/api/v1`)

Header	Value
`Content-Security-Policy`	`default-src 'none'; frame-ancestors 'none'; base-uri 'none'`
`Cache-Control`	`no-store`

Scoped to the `v1` group only — NOT applied to `/manager` (hashed assets keep their cacheable headers) nor to `/healthz`/`/readyz`.

Tier C — SPA document only (`spaDocumentCSP`, on the served index.html)

```
default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline';
img-src 'self' data:; font-src 'self'; connect-src 'self'; object-src 'none';
base-uri 'self'; form-action 'self'; frame-ancestors 'none'
[; upgrade-insecure-requests   — appended ONLY when TLS configured]
```

Set on the `/manager/` document, `/manager/index.html`, and the client-route fallback (`/manager/devices` etc.). Harmless on asset subresource responses (browsers apply CSP only to documents/workers).

---

3. HSTS TLS-gating — proof

`(*Server).tlsEnabled()` returns `cfg.TLSCertFile != "" && cfg.TLSKeyFile != ""` (matches `config.go:79-84` TLS-enablement). HSTS **and** the SPA CSP's `upgrade-insecure-requests` are both gated on it.

Test `TestSecurityHeaders_HSTS_TLSGated` (PASS): - plain-HTTP server (no cert/key) → `Strict-Transport-Security` **absent**. - TLS-configured server (`/tmp/cert.pem` + `/tmp/key.pem`) → HSTS present == `max-age=63072000; includeSubDomains`.

Why `upgrade-insecure-requests` is ALSO TLS-gated (evidence-based, not guessed): over plaintext HTTP on a non-localhost host, UIR upgrades the same-origin `/manager/assets/*` subresource loads to `https://…`, which fails against a plain-HTTP listener → **white-screen**. Emitting it only under TLS (a no-op hardening when the origin is already https) prevents that. Proven both ways by `TestSecurityHeaders_TierC_SPADocumentCSP` (plain HTTP → UIR absent) and `TestSecurityHeaders_TierC_UpgradeInsecureWhenTLS` (TLS → UIR present).

---

4. SPA-document CSP compatibility with the REAL built UI (§11.4.170/§11.4.123)

The embedded `manager-dist/` is a **REAL Vite/React production build** (not a placeholder): hashed assets `index-BCUfw0_g.js` (571 KB), `index-BhJ2of6B.css` (34 KB); Tailwind/shadcn markers (`bg-background font-sans antialiased`). Static analysis of the actual served bundle derived each directive:

Directive	Bundle requirement (captured fact)	Verdict
`script-src 'self'`	index.html loads ONE external module script `src="/assets/index-BCUfw0_g.js"` (same-origin). **No inline `<script>` body**, no `on*=` handlers, no `javascript:` URIs. Bundle grep: **zero** `eval(` / `new Function(` / `Function("` / `WebAssembly` / `.wasm`.	`'self'` sufficient; no hash/nonce/ `unsafe-inline`/ `unsafe-eval` needed
`style-src 'self' 'unsafe-inline'`	**REQUIRED** — bundle injects `<style>` at runtime: `createElement("style")` ×2 (React 19 style hoisting `rc(l,i.precedence)` + a CSS-in-JS injector `e.type="text/css"`), `.cssText`. Without `'unsafe-inline'` those blocks are blocked → unstyled UI.	widened to `'unsafe-inline'` (style only, per brief §1.3.3b)
`connect-src 'self'`	axios `baseURL="http://localhost:8080"` = the serving origin in the single-binary	`'self'` covers same-origin API calls

Directive	Bundle requirement (captured fact)	Verdict
	topology (SPA at `:8080/manager`, API at `:8080/api/v1`). No `new WebSocket` / `wss://`.	
`font-src 'self'`	CSS has **no** `@font-face` and **no** `url(...)` at all (Tailwind system-font stack).	`'self'` sufficient (no `data:` needed)
`img-src 'self' data:`	favicon `/vite.svg` (same-origin); no `data:image` in the bundle — `data:` kept as a safe superset.	compatible
worker/ `importScripts`	single `importScripts` occurrence is a dead feature-detection branch (`typeof self.importScripts=="function"` guarded by `WorkerGlobalScope`); **no `new Worker`**.	no `worker-src` needed

Widening required: exactly one — `style-src` gained `'unsafe-inline'` (the runtime `<style>` injection is real and load-bearing). No other directive was loosened; `script-src` stays the strict `'self'` (tightness proof: the bundle needs no `eval`, and the CSP grants none).

Router-level compatibility gate `TestSPACSP_CompatibleWithRealBundle` (PASS, embed built — NOT skipped) ties the served index.html to the emitted CSP: it asserts (1) every referenced subresource is same-origin `/assets/*`; (2) the CSP contains `style-src 'self' 'unsafe-inline'`; (3) the CSP does NOT contain `unsafe-eval`; (4) index.html carries no inline `<script>` body. All hold → the CSP permits exactly what the real UI needs and blocks the rest — it does NOT white-screen the built manager UI.

Honest boundary (§11.4.6): this proves the served CSP is COMPATIBLE with the built bundle's load requirements via static bundle analysis + router assertions. A live-browser console-zero-violations render (brief §5 step 2) is the complementary dynamic check; it is not runnable inside this Go test harness and is left to a browser-driver stream. The static evidence above is rock-solid for the load-path (script/style/ connect/font/img) the CSP governs.

5. Tier scoping — proof

`TestSecurityHeaders_TierB_ScopedToAPI` (PASS) + `TestSecurityHeaders_TierB_NotOnSPAAssets` (PASS): - `/api/v1/*` → `Content-Security-Policy: default-src 'none'…` + `Cache-Control: no-store`. - `/healthz` → NEITHER the API CSP NOR `no-store` (Tier A only). - `/manager/` document → the Tier-C SPA CSP, NOT the API CSP, NO `no-store`. - `/manager/assets/<hash>.js` → served 200, NOT `no-store` (hashed assets stay cacheable).

---

6. Test results (captured logs)

Logs: this directory — `test_security_headers_verbose.log`, `test_api_race_full.log`.

```
$ go build ./...                          → exit 0
$ go vet ./...                            → exit 0
$ gofmt -l <4 changed files>              → (empty — all formatted)

$ go test ./internal/api/ -run 'SecurityHeaders|SPACSP' -count=1
-v
--- PASS: TestSecurityHeaders_TierA_OnEveryResponseClass
--- PASS: TestSecurityHeaders_TierA_OnShed429
--- PASS: TestSecurityHeaders_HSTS_TLSGated
--- PASS: TestSecurityHeaders_TierB_ScopedToAPI
--- PASS: TestSecurityHeaders_TierB_NotOnSPAAssets
--- PASS: TestSecurityHeaders_TierC_SPADocumentCSP
--- PASS: TestSecurityHeaders_TierC_UpgradeInsecureWhenTLS
--- PASS: TestSPACSP_CompatibleWithRealBundle
PASS  ok  github.com/HelixDevelopment/helix_ota/server/internal/
api  0.012s

$ go test ./internal/api/... -race -count=1
ok  github.com/HelixDevelopment/helix_ota/server/internal/api
11.952s   (no data race; full suite incl. all pre-existing tests)
```

No pre-existing test regressed — the full `internal/api` suite (embed routing tests `TestManagerSPA_*`, rate-limit shed tests, handler/coverage/stress/chaos suites) passes with `-race` alongside the new tests. The embed routing behavior (§11.4.120 missing-asset-404, client-route fallback) is unchanged.

---

7. What was NOT changed (discipline)

- `HELIX_MAX_INFLIGHT` default untouched (0 = unlimited) — Item Q, operator chose to keep unlimited; out of this stream's scope.
- No capability removed (§11.4.122) — this only adds headers.
- Scope confined to `server/` (§11.4.119) — no edits to `clients/`, `dashboard/`, `submodules/`.
- No git write performed — conductor reviews + commits.